

← لطفاً این برگه را به همراه دفترچه پاسخ تحویل دهید →

نام و نام خانوادگی:

پرسش ۱ (۴ نمره) — تابع زیر را در نظر بگیرید. متد `Input::next_int` در صورتی که نتواند عدد بعدی را بخواند استثنای `InvalidDate` پرتاب می‌کند. سازنده `Date::Date` هم اگر مقادیر تاریخ نامعتبر باشند استثنای `InvalidDate` پرتاب می‌کند. تابع `next_date` را طوری تغییر دهید که اگر هر یک از فراخوانی‌های `Input::next_int` با استثنا مواجه شد استثنای `InputError` را پرتاب کند و اگر اعداد با موفقیت خوانده شدند اما تاریخ نامعتبر بود تاریخ اول فروردین ۱۴۰۲ را به عنوان نتیجه تابع برگرداند. فرض کنید سازنده `InputError` پارامتری نمی‌گیرد.

```
Date next_date(Input& inp) {
    int d = inp.next_int();
    int m = inp.next_int();
    return Date(d, m, 1402);
}
```

```
Date next_date(Input& inp) {
    try {
        int d = inp.next_int();
        int m = inp.next_int();
        return Date(d, m, 1402);
    } catch (NetworkEx nex) {
        throw InputError();
    } catch (InvalidDateEx iex) {
        return Date(1, 1, 1402);
    }
}
```

پرسش ۲ (۴ نمره) — در برنامه زیر به چهار خطای کامپایل اشاره کنید و به طور مختصر توضیح دهید:

<pre> ۱ #include <vector> ۲ #include <string> ۳ using namespace std; ۴ ۵ class Joghd { ۶ public: Constructor doesn't have return type ۷ void Joghd(string n, int a) ۸ : name(n) { age = a; } ۹ bool is_older_than(Joghd& j) { ۱۰ return age > j.age; ۱۱ } </pre>	<pre> ۱۲ private: ۱۳ string name; ۱۴ int age; ۱۵ }; ۱۶ ۱۷ int main() { No default constructor for Joghd ۱۸ vector<Joghd> joghds(10); ۱۹ Joghd j("Jooghd", 100); Wrong to take address of j ۲۰ if (joghds[0].is_older_than(&j)) ۲۱ return joghds[0].age(); ۲۲ } </pre>
---	---

No public method age()

پرسش ۳ (۴ نمره) — قرار است برنامه‌ای بنویسیم که اطلاعات دپارتمان‌ها، کارمندان و پروژه‌های یک شرکت را مدیریت کند. به این منظور سه کلاس Employee، Department و Project تعریف شده‌اند. می‌دانیم هر کارمند عضو یک و فقط یک دپارتمان است، در شرکت چندین پروژه در حال اجرا است و کارمندان می‌توانند در پروژه‌های مختلفی عضویت داشته باشند. هر دپارتمان اعضای خود را می‌شناسد و هر پروژه نیز اعضای خود را می‌شناسد. فرض کنید کلاس دپارتمان اعضای خود را در قالب یک بردار نگه می‌دارد. تایپ عناصر این بردار را کدامیک از این سه مورد تعریف می‌کنید؟ در مورد دلیل انتخاب خود و دلیل رد گزینه‌های دیگر مختصراً توضیح دهید.

- (a) Employee
- (b) Employee*
- (c) unique_ptr<Employee>
- (d) shared_ptr<Employee>

a مناسب نیست، به خاطر این که حذف یا اضافه کردن یک کارمند باعث تغییر آدرس کارمندهای دیگر می‌شود و اشاره‌گرهای احتمالی به آنها را تغییر می‌دهد.
b مناسب نیست، به خاطر این که آزادسازی حافظه کارمندا به طور خودکار انجام نمی‌شود.
c مناسب نیست، به خاطر این که اشیاء غیر از دپارتمان امکان داشتن اشاره‌گر به کارمندا را از دست می‌دهند.
d بهترین گزینه است، چون هیچ‌یک از مشکلات حالت‌های دیگر را ندارد.

پرسش ۴ (۴ نمره) — با داشتن کلاس‌های زیر نتیجه کامپایل و اجرای هر یک از قطعه‌کدهای زیر را مشخص کنید.

<pre>class A { public: virtual void f() { cout << 1; } virtual void g() { f(); } void h() { cout << 3; } virtual void i() = 0; }; class B : public A { public: virtual void f() { cout << 2; } void h() { f(); } }; class C : public B { public: virtual void i() { cout << 4; } virtual void f() { cout << 5; } }; class D : public C { public: virtual void i() { cout << 6; } virtual void g() { cout << 7; } };</pre>	(ب) <pre>B b; B* bp = &b; bp->f(); bp->g(); bp->h(); bp->i();</pre> Compile Error: B is abstract	(الف) <pre>C c; A* ap = &c; ap->f(); ap->g(); ap->h(); ap->i();</pre> 5534
	(د) <pre>C c; D* dp = &c; dp->f(); dp->g(); dp->h(); dp->i();</pre> Compile Error: Cannot cast C* into D*	(ج) <pre>C c; B* bp = &c; bp->f(); bp->g(); bp->h(); bp->i();</pre> 5554

پرسش ۵ (۴ نمره) — (الف) برای کلاس A که در زیر تعریف شده مخرب (destructor) و سازنده کپی (copy constructor) را به نحو صحیح پیاده‌سازی کنید. فرض کنید اندازه آرایه a همیشه با اندازه بردار v مساوی است.

```
class A {
public:
    A(int size) : v(size) {
        a = new int[size];
        count = 0;
    }
private:
    int count;
    int* a;
    vector<int> v;
};

A(const A& other) {
    v = other.v;
    count = other.count;
    a = new int[v.size()];
    for (int i = 0; i < v.size(); i++)
        a[i] = other.a[i];
}

~A() {
    delete[] a;
}
```

بارم: درست بودن پارامترهای سازنده کپی: ۰.۵ نمره
تخصیص حافظه برای a: ۰.۵ نمره
کپی کردن اعضای a: ۰.۵ نمره
استفاده از = برای کپی کردن v: ۰.۵ نمره
delete کردن a: ۰.۵ نمره
delete نکردن v: ۰.۵ نمره

(ب) تعریف عملگر = چه تفاوت‌هایی با سازنده کپی خواهد داشت؟ نیازی به پیاده‌سازی این عملگر نیست و صرفاً در مورد تفاوت‌ها توضیح دهید. لزوم چک کردن self assignment: ۰.۵ نمره
آزاد کردن حافظه a قبل از تخصیص حافظه جدید: ۰.۵ نمره